CS-308 Data Science Practical – Master Answer Book (Selected Slips)

Oct 14, 2026 · @Faisal

How to use this book

Every selected slip has its Q1, Q2 and Q2-OR program, followed by likely viva questions with short answers. Code follows the same style as your teacher's solutions (pandas, sklearn, matplotlib, mlxtend), with a few fixes where the teacher's code would not run on the real dataset.

Install everything once before the exam:

```
pip install pandas numpy matplotlib seaborn scikit-learn scipy mlxtend
matplotlib-venn wordcloud squarify plotly
```

Slip	PDF page	Q1	Q2	Q2 OR
4	6	Pie chart (payment)	Logistic Regression (Gaming)	K-Means (E-commerce)
5	7	Label Encoding	Netflix/Prime Venn + Pie	KNN (Spambase)
6	8	Scatter plot (cars)	Linear Regression (Sports)	KNN (Mall Customers)
7	9	Bar chart (courses)	KNN (Student dataset)	Apriori (Groceries)
11	13	Bar chart (departments)	Logistic Regression (Smartphone)	K-Means (E-commerce)
14	16	Dendrogram	Polynomial Regression (Car)	Merge + Label + One-Hot
15	17	Equal Frequency Binning	Treemap + 3D Scatter	Apriori (Groceries)
16	18	Correlation Heat Map	Min-Max, Standard, Normalize	Apriori (Student activity)

Slip	PDF page	Q1	Q2	Q2 OR
17	19	Data Integration	Treemap + 3D Scatter	Random Forest (Student Grading)
18	20	Data Cleaning + IQR	World Cities Map + Treemap	Linear Regression (Crop)
19	21	Box Plot + Outliers	Min-Max, Standard, Normalize	Random Forest (Heart)
20	22	Standardization	Books Venn + Word Cloud	Apriori (Student activity)
22	24	Normalization	Netflix/Prime Venn + Pie	K-Means (Attendance, Marks)
23	25	Dendrogram with names	Min-Max, Standard, Normalize	KNN (Student Grading)
25	27	Bar chart (courses)	Merge + Label + One-Hot	Apriori (Groceries)

Three habits that save marks in the exam:

1. Right after `pd.read_csv(...)`, always run `print(df.head())`, `print(df.info())` and `print(df.columns)`. If a column name in this book differs slightly from the file you get, just change the name in the code.
2. Keep `random_state=42` everywhere so your output stays the same when you re-run in front of the external.
3. If a program repeats from an earlier slip, this book says so and you can copy that code as it is.

Slip 4 (Page 6)

Q1. Pie Chart – Preferred Payment Method [10]

```
import matplotlib.pyplot as plt

payment_methods = ['UPI', 'Credit Card', 'Debit Card', 'Cash', 'Net Banking']
customers = [150, 80, 60, 40, 30]
colors = ['gold', 'skyblue', 'lightgreen', 'salmon', 'violet']
explode = [0.1, 0, 0, 0, 0]          # pull out the biggest slice (UPI)

plt.figure(figsize=(7, 7))
plt.pie(customers, labels=payment_methods, colors=colors,
        autopct='%1.1f%%', explode=explode, startangle=90, shadow=True)
plt.title('Preferred Payment Methods Among Customers')
plt.axis('equal')                     # keeps the pie a perfect circle
plt.show()
```

Output: UPI 41.7%, Credit Card 22.2%, Debit Card 16.7%, Cash 11.1%, Net Banking 8.3% (total 360 customers).

Q2. Logistic Regression – Online Gaming (Casual vs Professional) [20]

The real dataset has no Casual/Professional column. Its columns are PlayTimeHours, InGamePurchases, PlayerLevel, AchievementsUnlocked, SessionsPerWeek, AvgSessionDurationMinutes and EngagementLevel (Low/Medium/High). So we create the label: High engagement = Professional, others = Casual. Tell the external this clearly.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix,
ConfusionMatrixDisplay)

# 1. Read and inspect
df = pd.read_csv('online_gaming_behavior_dataset.csv')
print(df.head())
print(df.info())
print(df.isnull().sum())
```

```
# 2. Preprocess
df = df.dropna().drop_duplicates()
df['Player_Type'] = df['EngagementLevel'].apply(
    lambda x: 'Professional' if x == 'High' else 'Casual')
print(df['Player_Type'].value_counts())

features = ['PlayTimeHours', 'AchievementsUnlocked', 'InGamePurchases',
            'PlayerLevel', 'SessionsPerWeek', 'AvgSessionDurationMinutes']
X = df[features]
y = df['Player_Type']

# 3. Split and scale
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Train
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
pred = model.predict(X_test)

# 5. Evaluate
print('Accuracy :', accuracy_score(y_test, pred))
print('Precision:', precision_score(y_test, pred, pos_label='Professional'))
print('Recall   :', recall_score(y_test, pred, pos_label='Professional'))
print('F1 Score :', f1_score(y_test, pred, pos_label='Professional'))
cm = confusion_matrix(y_test, pred)
print('Confusion Matrix:\n', cm)

ConfusionMatrixDisplay(cm, display_labels=model.classes_).plot()
plt.title('Confusion Matrix - Gaming Players')
plt.show()
```

Q2 OR. K-Means – E-commerce Customers + Interpretation [20]

Columns in the file: Email, Address, Avatar (text) and Avg. Session Length, Time on App, Time on Website, Length of Membership, Yearly Amount Spent (numbers). We use only the numeric ones.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# 1. Read and inspect
df = pd.read_csv('Ecommerce Customers.csv')
print(df.head())
print(df.info())
print(df.isnull().sum())

# 2. Handle missing values + select numeric features
df = df.fillna(df.mean(numeric_only=True))
features = ['Avg. Session Length', 'Time on App', 'Time on Website',
            'Length of Membership', 'Yearly Amount Spent']
scaler = StandardScaler()
X = scaler.fit_transform(df[features])

# 3. Elbow method to justify K
inertia = []
for k in range(1, 11):
    inertia.append(KMeans(n_clusters=k, random_state=42,
n_init=10).fit(X).inertia_)
plt.plot(range(1, 11), inertia, marker='o')
plt.xlabel('K'); plt.ylabel('Inertia'); plt.title('Elbow Method')
plt.show()

# 4. Apply K-Means with K = 3
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X)

# 5. Interpret: average of each feature in each cluster
summary = df.groupby('Cluster')[features].mean()
print(summary)

order = summary['Yearly Amount Spent'].sort_values().index
names = dict(zip(order, ['Low Value', 'Medium Value', 'High Value']))
df['Segment'] = df['Cluster'].map(names)
print(df['Segment'].value_counts())

# 6. Plot with centroids (back in original units)
centers = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(df['Length of Membership'], df['Yearly Amount Spent'],
```

```
            c=df['Cluster'], cmap='viridis')
plt.scatter(centers[:, 3], centers[:, 4], c='red', marker='X', s=200,
label='Centroids')
plt.xlabel('Length of Membership (years)')
plt.ylabel('Yearly Amount Spent')
plt.title('E-commerce Customer Segments')
plt.legend()
plt.show()
```

Interpretation to write: the High Value cluster has the longest membership and highest yearly spend, so the company should reward them (loyalty offers). The Low Value cluster has short membership and low spend, so they need discounts to stay. Length of Membership is the feature that separates the clusters most.

Slip 4 – Viva

1. **When do we use a pie chart?** To show parts of a whole (percentages). It works best with few categories (5–6).
2. **What does `autopct='%1.1f%%'` do?** It prints each slice's percentage with 1 decimal place on the chart.
3. **Logistic Regression is called regression, so why is it classification?** It predicts a probability (0 to 1) using the sigmoid function, then a threshold of 0.5 turns it into a class.
4. **Sigmoid formula?** σ(z) = 1 / (1 + e^(−z)). It squeezes any number into the range 0–1.
5. **Precision vs Recall?** Precision = TP / (TP + FP): of predicted Professionals, how many were right. Recall = TP / (TP + FN): of real Professionals, how many we caught.
6. **F1 score?** Harmonic mean of precision and recall: 2PR / (P + R). Useful when classes are unbalanced.
7. **Why StandardScaler?** Features have different units (hours vs level); scaling gives each mean 0 and std 1 so none dominates.
8. **Why `stratify=y`?** Keeps the same class ratio in train and test sets.
9. **Is K-Means supervised?** No, it is unsupervised. It has no target label; it groups similar points.
10. **Steps of K-Means?** Pick K centroids → assign each point to nearest centroid → recompute centroids as cluster means → repeat until centroids stop moving.
11. **How do you choose K?** Elbow method: plot inertia (within-cluster sum of squares) vs K and pick the bend point.

12. **Why scale before K-Means?** K-Means uses Euclidean distance, so large-valued features would dominate without scaling.

Slip 5 (Page 7)

Q1. Label Encoding – customer_DataFrame [10]

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

customer_df = pd.DataFrame({
    'Customer_ID': [101, 102, 103, 104, 105, 106],
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female', 'Male'],
    'City': ['Nashik', 'Pune', 'Mumbai', 'Nashik', 'Pune', 'Mumbai'],
    'Membership_Type': ['Gold', 'Silver', 'Gold', 'Bronze', 'Silver',
'Bronze']
})
print('Before Encoding:')
print(customer_df)

encoded_df = customer_df.copy()
for col in ['Gender', 'City', 'Membership_Type']:
    le = LabelEncoder()                      # new encoder for each column
    encoded_df[col] = le.fit_transform(encoded_df[col])
    print(col, 'mapping:', dict(zip(le.classes_, le.transform(le.classes_))))

print('\nAfter Encoding:')
print(encoded_df)
```

LabelEncoder sorts categories alphabetically, so Female = 0, Male = 1 and Mumbai = 0, Nashik = 1, Pune = 2. Customer_ID is not encoded because it is already numeric and only an identifier.

Q2. Netflix & Amazon Prime – Venn Diagram + Pie Chart [20]

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib_venn import venn2

# 1. Generate DataFrame
df = pd.DataFrame({
```

```
    'Customer_ID': range(1, 13),
    'Name': ['Amit', 'Riya', 'Sahil', 'Neha', 'Om', 'Pooja',
             'Karan', 'Sneha', 'Rohan', 'Tanvi', 'Yash', 'Isha'],
    'Netflix':      ['Yes', 'Yes', 'Yes', 'No', 'Yes', 'No',
                     'Yes', 'No', 'Yes', 'No', 'Yes', 'No'],
    'Amazon_Prime': ['No', 'Yes', 'No', 'Yes', 'Yes', 'Yes',
                     'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes']
})
print(df)

# 2. Make sets of customers
netflix = set(df[df['Netflix'] == 'Yes']['Name'])
prime = set(df[df['Amazon_Prime'] == 'Yes']['Name'])

only_netflix = netflix - prime
only_prime = prime - netflix
both = netflix & prime
print('Only Netflix:', only_netflix)
print('Only Prime  :', only_prime)
print('Both        :', both)

# 3. Venn diagram
plt.figure(figsize=(6, 6))
venn2([netflix, prime], set_labels=('Netflix', 'Amazon Prime'),
      set_colors=('red', 'skyblue'), alpha=0.7)
plt.title('Netflix vs Amazon Prime Users')
plt.show()

# 4. Pie chart
sizes = [len(only_netflix), len(only_prime), len(both)]
labels = ['Only Netflix', 'Only Amazon Prime', 'Both']
plt.figure(figsize=(6, 6))
plt.pie(sizes, labels=labels, colors=['red', 'skyblue', 'mediumpurple'],
        autopct='%1.1f%%', startangle=90)
plt.title('Customer Distribution by Streaming Service')
plt.show()
```

Q2 OR. KNN – Spambase (Spam / Not Spam) [20]

Spambase has 57 numeric features (word and character frequencies, capital-letter run lengths) and one 0/1 label column. The label name differs between copies of the file, so the code finds it automatically.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix,
classification_report)

# 1. Read and inspect
df = pd.read_csv('spambase.csv')
print(df.shape)
print(df.columns)
print(df.isnull().sum().sum(), 'missing values')

# 2. Preprocess
df = df.dropna().drop_duplicates()
names = [c for c in df.columns if c.lower() in ('spam', 'class', 'label',
'is_spam', 'target')]
target = names[0] if names else df.columns[-1]
print('Target column:', target)
print(df[target].value_counts())

X = df.drop(columns=[target])
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 3. Train KNN (k = 5)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
pred = knn.predict(X_test)

# 4. Evaluate
print('Accuracy :', accuracy_score(y_test, pred))
print('Precision:', precision_score(y_test, pred))
```

```
print('Recall   :', recall_score(y_test, pred))
print('F1 Score :', f1_score(y_test, pred))
print('Confusion Matrix:\n', confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, target_names=['Not Spam', 'Spam']))
```

Expected accuracy is around 0.89–0.92.

Slip 5 – Viva

1. **What is Label Encoding?** Converting each category into an integer (0, 1, 2…) so ML models can use it.
2. **Label Encoding vs One-Hot Encoding?** Label gives one column of numbers and suggests an order; One-Hot makes a 0/1 column per category with no order. Use Label for ordinal or target data, One-Hot for nominal features.
3. **Problem with label encoding City?** The model may think Pune (2) > Mumbai (0), which is meaningless. One-Hot avoids this.
4. **What is a Venn diagram?** Overlapping circles showing sets and their intersections; the overlap here = customers using both services.
5. **Which library draws Venn diagrams?** `matplotlib_venn`, with `venn2` for 2 sets and `venn3` for 3 sets.
6. **Set operations used?** Difference (`-`) for only-one-service, intersection (`&`) for both, union (`|`) for total.
7. **How does KNN classify?** It finds the K nearest training points (Euclidean distance) and takes a majority vote of their labels.
8. **Why is KNN called a lazy learner?** It does no training; it only stores data and does all work at prediction time.
9. **How to choose K?** Try several values and pick the one with best test accuracy; use an odd K to avoid ties. Small K overfits, large K underfits.
10. **Why is recall important in spam detection?** Precision matters more if you don't want real mail sent to spam; recall matters if you want to catch all spam. F1 balances both.

Slip 6 (Page 8)

Q1. Scatter Plot – Engine Size vs Fuel Efficiency [10]

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    'Car_Model': ['Alto', 'Swift', 'City', 'Creta', 'XUV700', 'Fortuner'],
    'Engine_Size': [0.8, 1.2, 1.5, 1.5, 2.2, 2.8],          # litres
    'Fuel_Efficiency': [24.0, 22.5, 18.0, 17.0, 13.0, 10.0] # km/L
})
print(df)

colors = ['red', 'blue', 'green', 'orange', 'purple', 'brown']
plt.figure(figsize=(8, 5))
plt.scatter(df['Engine_Size'], df['Fuel_Efficiency'], c=colors, s=120)

for i in range(len(df)):                    # write model name near each
point
    plt.annotate(df['Car_Model'][i], (df['Engine_Size'][i] + 0.03,
df['Fuel_Efficiency'][i]))

plt.title('Engine Size vs Fuel Efficiency')
plt.xlabel('Engine Size (litres)')
plt.ylabel('Fuel Efficiency (km/L)')
plt.grid(True)
plt.show()
```

Conclusion: negative relationship — bigger engines give lower mileage.

Q2. Simple Linear Regression – Sports Performance [20]

The file `collegiate_athlete_injury_dataset.csv` has columns like Training_Hours_Per_Week and Performance_Score. The code finds them by keyword, so small spelling differences do not break it.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

```
# 1. Read and inspect
df = pd.read_csv('collegiate_athlete_injury_dataset.csv')
print(df.head())
print(df.columns)

xcol = [c for c in df.columns if 'training' in c.lower() and 'hour' in
c.lower()][0]
ycol = [c for c in df.columns if 'performance' in c.lower()][0]
print('X =', xcol, '| y =', ycol)

# 2. Preprocess
df = df[[xcol, ycol]].dropna()
X = df[[xcol]]
y = df[ycol]

# 3. Split, train
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
print('Slope (m)     :', model.coef_[0])
print('Intercept (c) :', model.intercept_)

# 4. Evaluate
pred = model.predict(X_test)
print('R2 Score:', r2_score(y_test, pred))

# 5. Plot
plt.scatter(X, y, color='blue', alpha=0.6, label='Actual data')
plt.plot(X, model.predict(X), color='red', linewidth=2, label='Regression
line')
plt.xlabel('Training Hours Per Week')
plt.ylabel('Performance Score')
plt.title('Training Hours vs Performance Score')
plt.legend()
plt.show()
```

If R^2 comes out low (close to 0), say: training hours alone explain little of performance; other factors such as fatigue and recovery matter too. That is a valid conclusion.

Q2 OR. KNN – Customer (Mall) Dataset Segments [20]

Mall_Customers.csv has CustomerID, Gender (sometimes spelled Genre), Age, Annual Income (k$) and Spending Score (1-100), but no segment label. So first K-Means creates 5 segments, then KNN learns to classify customers into those segments.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.cluster import KMeans
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

# 1. Read and preprocess
df = pd.read_csv('Mall_Customers.csv')
print(df.head())
print(df.isnull().sum())
df = df.dropna()
df = df.rename(columns={'Genre': 'Gender'})
df['Gender'] = LabelEncoder().fit_transform(df['Gender'])   # Female=0,
Male=1

features = ['Annual Income (k$)', 'Spending Score (1-100)']
scaler = StandardScaler()
X = scaler.fit_transform(df[features])

# 2. Create segment labels
df['Segment'] = KMeans(n_clusters=5, random_state=42,
n_init=10).fit_predict(X)
print(df['Segment'].value_counts())
y = df['Segment']

# 3. Train KNN
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
pred = knn.predict(X_test)

# 4. Evaluate
print('Accuracy:', accuracy_score(y_test, pred))
```

```
cm = confusion_matrix(y_test, pred)
print('Confusion Matrix:\n', cm)
ConfusionMatrixDisplay(cm).plot()
plt.title('KNN - Customer Segments')
plt.show()

# 5. Classify a new customer
new = scaler.transform(pd.DataFrame([[70, 85]], columns=features))
print('New customer segment:', knn.predict(new)[0])
```

Slip 6 – Viva

1. **What does a scatter plot show?** The relationship (positive, negative or none) between two numeric variables.
2. **What is correlation here?** Strong negative: as engine size goes up, fuel efficiency goes down.
3. **Equation of Simple Linear Regression?** y = mx + c, where m = slope and c = intercept. Here Performance = m × Training Hours + c.
4. **How does Linear Regression find the best line?** Ordinary Least Squares: it minimises the sum of squared errors between actual and predicted values.
5. **What is R² score?** Fraction of variation in y explained by the model; 1 = perfect, 0 = no better than the mean.
6. **Simple vs Multiple Linear Regression?** Simple has one input feature; Multiple has two or more.
7. **Why did you use K-Means before KNN?** The Mall dataset has no class label. KNN is supervised and needs one, so K-Means created segment labels first.
8. **What distance does KNN use?** Euclidean by default: $\sqrt{((x1-x2)^2 + (y1-y2)^2)}$.
9. **Why test_size=0.2?** 80% data to train, 20% kept unseen to test the model fairly.
10. **What is the Mall dataset's typical segments?** High income–high spending (target customers), high income–low spending (careful), low income–high spending (careless), low–low (sensible), and average customers.

Slip 7 (Page 9)

Q1. Bar Chart – Students Enrolled in Courses [10]

```
import matplotlib.pyplot as plt

courses = ['BCA', 'BBA', 'B.Com', 'B.Sc', 'BA']
students = [60, 45, 75, 50, 35]
colors = ['red', 'blue', 'green', 'orange', 'purple']

plt.figure(figsize=(8, 5))
bars = plt.bar(courses, students, color=colors, edgecolor='black')

for bar in bars:                                    # value on top of each bar
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 1,
             bar.get_height(), ha='center')

plt.title('Number of Students Enrolled in Each Course')
plt.xlabel('Courses')
plt.ylabel('Number of Students')
plt.show()
```

Q2. KNN – Student Dataset (Performance Categories) [20]

The exact columns of `student_dataset_v2.csv` vary between copies, so this code finds the target by keyword. If the target is a numeric score, it converts it into Low / Average / High.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

# 1. Read and inspect
df = pd.read_csv('student_dataset_v2.csv')
print(df.head())
print(df.info())
print(df.isnull().sum())
```

```
# 2. Handle missing values and duplicates
for c in df.columns:
    if df[c].dtype == 'object':
        df[c] = df[c].fillna(df[c].mode()[0])
    else:
        df[c] = df[c].fillna(df[c].median())
df = df.drop_duplicates()

# 3. Remove ID / name columns (they do not help prediction)
drop = [c for c in df.columns if c.lower().endswith('id') or 'name' in
c.lower()]
df = df.drop(columns=drop)

# 4. Choose target column
keys = ['grade', 'performance', 'result', 'category']
target = [c for c in df.columns if any(k in c.lower() for k in keys)][0]
if df[target].dtype != 'object' and df[target].nunique() > 10:
    df[target] = pd.cut(df[target], bins=3, labels=['Low', 'Average',
'High'])
print('Target:', target)
print(df[target].value_counts())

# 5. Encode text features
y = df[target].astype(str)
X = df.drop(columns=[target])
for c in X.select_dtypes(include='object').columns:
    X[c] = LabelEncoder().fit_transform(X[c])

# 6. Split, scale, train
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
pred = knn.predict(X_test)

# 7. Evaluate
print('Accuracy:', accuracy_score(y_test, pred))
cm = confusion_matrix(y_test, pred)
```

```
print('Confusion Matrix:\n', cm)
ConfusionMatrixDisplay(cm, display_labels=knn.classes_).plot()
plt.title('KNN - Student Performance')
plt.show()
```

If the keyword search finds nothing, look at `df.columns` and set `target = '<column name>'` by hand.

Q2 OR. Apriori – Groceries Market Basket (support 0.3 and 0.4) [20]

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# 1. Create and explore the dataset
data = pd.DataFrame({
    'S.No': [1, 2, 3, 4, 5, 6, 7],
    'Items Purchased': [
        'Basmati Rice, Toor Dal, Turmeric Powder, Cooking Oil',
        'Atta, Sugar, Tea, Milk',
        'Rice, Moong Dal, Salt, Cooking Oil',
        'Wheat Flour, Chickpeas, Turmeric Powder, Cumin Seeds',
        'Poha, Peanuts, Onions, Cooking Oil',
        'Basmati Rice, Paneer, Garam Masala, Tomatoes',
        'Atta, Potatoes, Onions, Cooking Oil']
})
print(data)
print(data.isnull().sum())

# 2. Preprocess: drop missing rows, split text into item lists
data = data.dropna()
transactions = data['Items Purchased'].apply(
    lambda s: [item.strip() for item in s.split(',')]).tolist()

# 3. Convert to one-hot (True/False) format
te = TransactionEncoder()
df = pd.DataFrame(te.fit_transform(transactions), columns=te.columns_)
print(df.astype(int))
print('Item support:\n', df.mean().sort_values(ascending=False))

# 4. Apriori with different support values
def run_apriori(support):
```

```
    print('\n===== Minimum Support =', support, '=====')
    freq = apriori(df, min_support=support, use_colnames=True)
    print('Frequent Itemsets:\n', freq)
    if len(freq) == 0 or freq['itemsets'].apply(len).max() < 2:
        print('No association rules: no itemset with 2 or more items is
frequent.')
        return
    try:
        rules = association_rules(freq, num_itemsets=len(df),
                                  metric='confidence', min_threshold=0.5)
    except TypeError:        # older mlxtend versions
        rules = association_rules(freq, metric='confidence',
min_threshold=0.5)
    print(rules[['antecedents', 'consequents', 'support', 'confidence',
'lift']])

for s in [0.3, 0.4]:
    run_apriori(s)

run_apriori(0.2)   # extra: lower support to show rules actually appear
```

Expected result (be ready to explain this): Cooking Oil is in 4 of 7 baskets, support 0.57, so it is the only frequent item at both 0.3 and 0.4. Every other item appears at most twice (2/7 = 0.29 < 0.3), so no rules are produced. At support 0.2, pairs like {Onions, Cooking Oil} become frequent and you get the rule Onions → Cooking Oil with confidence 1.0.

Slip 7 – Viva

1. **Bar chart vs histogram?** Bar chart compares categories (gaps between bars); a histogram shows the distribution of numeric data in bins (no gaps).
2. **Why use KNN for classification here?** It is simple, needs no assumptions about data, and works well on small, scaled numeric data.
3. **Why `fillna` instead of `dropna`?** Dropping rows loses data; filling with median (numbers) or mode (text) keeps all records.
4. **What is market basket analysis?** Finding products that are bought together, to plan offers and shelf placement.
5. **Support?** Fraction of transactions containing the itemset. support(Oil) = 4/7 = 0.57.
6. **Confidence?** confidence(A→B) = support(A∪B) / support(A): how often B is bought when A is bought.
7. **Lift?** confidence(A→B) / support(B). Lift > 1 means A and B are positively related.

8. **Apriori property?** Every subset of a frequent itemset must also be frequent, so infrequent items are pruned early.
9. **Why does support 0.4 give fewer itemsets than 0.3?** Higher minimum support is a stricter filter, so fewer itemsets pass.
10. **What does TransactionEncoder do?** Converts lists of items into a True/False table with one column per item — the numeric format Apriori needs.

Slip 11 (Page 13)

Q1. Bar Chart – Employees by Department [10]

```
import matplotlib.pyplot as plt

departments = ['HR', 'Finance', 'IT', 'Sales', 'Marketing']
employees = [25, 30, 60, 45, 35]

plt.figure(figsize=(8, 5))
bars = plt.bar(departments, employees,
               color=['teal', 'orange', 'royalblue', 'green', 'crimson'])
for bar in bars:
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 1,
             bar.get_height(), ha='center')

plt.title('Distribution of Employees Among Departments')
plt.xlabel('Department')
plt.ylabel('Number of Employees')
plt.show()
```

Q2. Logistic Regression – Smartphone Price Categories [20]

`train.csv` has 20 specification columns (battery_power, ram, int_memory, pc, fc, px_height, px_width, talk_time, etc.) and target `price_range`: 0 = low, 1 = medium, 2 = high, 3 = very high.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, f1_score, confusion_matrix,
```

```
                              ConfusionMatrixDisplay, classification_report)

# 1. Read and inspect
df = pd.read_csv('train.csv')
print(df.head())
print(df.info())
print(df.isnull().sum().sum(), 'missing values')

# 2. Preprocess
df = df.dropna().drop_duplicates()
X = df.drop(columns=['price_range'])
y = df['price_range']
print(y.value_counts())          # 500 phones in each class (balanced)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 3. Train (multi-class logistic regression)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
pred = model.predict(X_test)

# 4. Evaluate
print('Accuracy:', accuracy_score(y_test, pred))
print('F1 Score (weighted):', f1_score(y_test, pred, average='weighted'))
cm = confusion_matrix(y_test, pred)
print('Confusion Matrix:\n', cm)
print(classification_report(y_test, pred))

ConfusionMatrixDisplay(cm, display_labels=['Low', 'Medium', 'High', 'Very
High']).plot()
plt.title('Smartphone Price Category - Confusion Matrix')
plt.show()
```

Expected accuracy is about 0.95–0.97. RAM is the strongest feature for price.

Q2 OR. K-Means – E-commerce Customers (display clusters) [20]

Same program as **Slip 4 – Q2 OR**. Copy it as it is; it reads the data, checks missing values, selects the 5 numeric features, applies K-Means with K = 3, and displays the clusters with centroids. To show more views of the clusters, add this at the end:

```
import seaborn as sns
sns.pairplot(df[features + ['Cluster']], hue='Cluster', palette='viridis')
plt.show()
```

Slip 11 – Viva

1. **Why `plt.figure(figsize=(8,5))`?** Sets chart width and height in inches.
2. **How does Logistic Regression handle 4 classes?** sklearn uses multinomial (softmax) logistic regression, which gives a probability for each class and picks the highest.
3. **Why `average='weighted'` in F1?** For multi-class, it averages F1 of each class weighted by number of samples in that class.
4. **What is `max_iter=1000`?** Maximum iterations for the solver to converge; the default 100 may show a convergence warning.
5. **How to read a confusion matrix?** Rows = actual class, columns = predicted class. Diagonal = correct predictions; off-diagonal = mistakes.
6. **Is this dataset balanced?** Yes, 500 phones per class, so accuracy is a fair metric.
7. **What is a centroid?** The mean point of all members of a cluster.
8. **What is inertia?** Sum of squared distances of points to their cluster centroid. Lower = tighter clusters.
9. **Limitations of K-Means?** You must choose K, it assumes round clusters of similar size, and outliers pull centroids.
10. **fit vs transform vs fit_transform?** fit learns parameters (mean, std), transform applies them, fit_transform does both. We fit only on training data to avoid data leakage.

Slip 14 (Page 16)

Q1. Dendrogram – Grouping Similar Products [10]

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.preprocessing import StandardScaler

df = pd.DataFrame({
    'Product': ['Phone', 'Laptop', 'Earbuds', 'Tablet', 'Smartwatch',
'Speaker'],
    'Price': [15000, 55000, 2000, 25000, 8000, 3500],
    'Rating': [4.3, 4.6, 4.0, 4.4, 4.1, 3.9],
    'Number_of_Sales': [900, 300, 1500, 400, 700, 1200]
})
print(df)

# Scale so Price (thousands) does not dominate Rating (1-5)
X = StandardScaler().fit_transform(df[['Price', 'Rating',
'Number_of_Sales']])

Z = linkage(X, method='ward')
plt.figure(figsize=(8, 5))
dendrogram(Z, labels=df['Product'].tolist())
plt.title('Dendrogram - Similar Products')
plt.xlabel('Products')
plt.ylabel('Euclidean Distance')
plt.show()
```

Reading it: products joined at a low height are similar (for example Earbuds and Speaker: cheap, high sales). Cutting the tree with a horizontal line gives the groups.

Q2. Polynomial Regression – Car Price Prediction [20]

The real `car data.csv` columns are Car_Name, Year, Selling_Price, Present_Price, Kms_Driven, Fuel_Type, Seller_Type, Transmission, Owner. There is no Vehicle Age column, so we compute it from Year. (The teacher's code uses names like 'Vehicle Age' and 'Kms Driven' that do not exist in the file and would give a KeyError.)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from datetime import datetime
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# 1. Read and preprocess
df = pd.read_csv('car data.csv')
print(df.head())
print(df.isnull().sum())
df = df.dropna().drop_duplicates()
df['Vehicle_Age'] = datetime.now().year - df['Year']

X = df[['Vehicle_Age', 'Present_Price', 'Kms_Driven']]
y = df['Selling_Price']

# 2. Split + polynomial features (degree 2)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

# 3. Train and evaluate
model = LinearRegression()
model.fit(X_train_poly, y_train)
pred = model.predict(X_test_poly)
print('R2 Score:', r2_score(y_test, pred))

# 4a. Actual vs predicted
plt.scatter(y_test, pred, color='blue')
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
plt.xlabel('Actual Selling Price (lakhs)')
plt.ylabel('Predicted Selling Price (lakhs)')
plt.title('Actual vs Predicted Selling Price')
plt.show()

# 4b. Fitted polynomial curve (Present_Price vs Selling_Price)
poly1 = PolynomialFeatures(degree=2)
xp = df[['Present_Price']]
```

```
curve_model = LinearRegression().fit(poly1.fit_transform(xp), y)
x_range = pd.DataFrame({'Present_Price':
np.linspace(xp['Present_Price'].min(), xp['Present_Price'].max(), 100)})
plt.scatter(xp, y, color='blue', alpha=0.6, label='Actual data')
plt.plot(x_range, curve_model.predict(poly1.transform(x_range)),
         color='red', linewidth=2, label='Polynomial curve (degree 2)')
plt.xlabel('Present Price (lakhs)')
plt.ylabel('Selling Price (lakhs)')
plt.title('Polynomial Regression - Car Price')
plt.legend()
plt.show()
```

Q2 OR. Data Integration + Label Encoding + One-Hot Encoding (Mobiles) [20]

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df1 = pd.DataFrame({
    'Brand': ['Apple', 'Samsung', 'OnePlus', 'Vivo', 'Xiaomi'],
    'Model': ['iPhone 15', 'Galaxy S24', 'OnePlus 12', 'V30', 'Redmi Note
13'],
    'Price': [79900, 74999, 64999, 33999, 17999],
    'RAM': [6, 8, 12, 8, 6]
})
df2 = pd.DataFrame({
    'Model': ['iPhone 15', 'Galaxy S24', 'OnePlus 12', 'V30', 'Redmi Note
13'],
    'Storage': [128, 256, 256, 128, 128],
    'Battery_Capacity': [3349, 4000, 5400, 5000, 5000],
    'Operating_System': ['iOS', 'Android', 'Android', 'Android', 'Android']
})
print('DataFrame 1:\n', df1)
print('\nDataFrame 2:\n', df2)

# a) Data Integration
merged = pd.merge(df1, df2, on='Model', how='inner')
print('\nMerged DataFrame:\n', merged)

# b) Label Encoding on Brand
transformed = merged.copy()
le = LabelEncoder()
transformed['Brand'] = le.fit_transform(transformed['Brand'])
```

```
print('\nBrand mapping:', dict(zip(le.classes_, le.transform(le.classes_))))

# c) One-Hot Encoding on Operating_System
transformed = pd.get_dummies(transformed, columns=['Operating_System'],
dtype=int)
print('\nTransformed DataFrame:\n', transformed)
```

Slip 14 – Viva

1. **What is a dendrogram?** A tree diagram showing how hierarchical clustering merges points step by step; height = distance at which they merge.
2. **Agglomerative vs divisive clustering?** Agglomerative is bottom-up (each point starts alone and merges); divisive is top-down (all in one cluster and splits).
3. **What is the `ward` method?** It merges the pair of clusters that causes the least increase in within-cluster variance. Others: single, complete, average.
4. **How do you get number of clusters from a dendrogram?** Draw a horizontal line across the longest vertical gap; the number of lines it cuts = number of clusters.
5. **What is Polynomial Regression?** Linear regression on powers of features (x, x^2, x1·x2…) so it can fit a curve.
6. **Is Polynomial Regression linear or non-linear?** Non-linear in x, but linear in the coefficients, so LinearRegression is still used.
7. **What does PolynomialFeatures(degree=2) do with 3 features?** Creates 10 columns: 1, the 3 originals, their 3 squares, and 3 pairwise products.
8. **Risk of high degree?** Overfitting: the curve follows noise in training data and performs badly on test data.
9. **What is Data Integration?** Combining data from multiple sources into one dataset, here using `pd.merge` on the common key Model.
10. **inner vs outer vs left join?** Inner keeps only matching keys; outer keeps all keys from both; left keeps all from the left DataFrame.
11. **Why One-Hot for Operating_System?** OS is nominal (no order), so each OS gets its own 0/1 column.

Slip 15 (Page 17)

Q1. Equal Frequency Binning – age_records [10]

```
import pandas as pd

age_records = pd.DataFrame({
    'Person_ID': range(1, 13),
    'Age': [18, 21, 23, 26, 29, 33, 37, 41, 46, 52, 58, 65]
})
print('Original Data:')
print(age_records)

# 4 bins, each with the same number of records (12 / 4 = 3)
age_records['Age_Bin'], edges = pd.qcut(
    age_records['Age'], q=4,
    labels=['Young', 'Adult', 'Middle-aged', 'Senior'], retbins=True)

print('\nBin edges:', edges)
print('\nDiscretized Data:')
print(age_records)
print('\nRecords per bin:')
print(age_records['Age_Bin'].value_counts().sort_index())
```

Each bin gets exactly 3 people, but the bin widths are different. That is the key difference from equal width binning.

Q2. Treemap + 3D Scatter Plot – City Data [20]

```
import pandas as pd
import matplotlib.pyplot as plt
import squarify

df = pd.DataFrame({
    'City': ['Mumbai', 'Pune', 'Nashik', 'Nagpur', 'Aurangabad', 'Kolhapur'],
    'Area (sq. km)': [603, 331, 267, 228, 139, 67],
    'Average Temperature (C)': [27.5, 25.0, 24.5, 27.0, 26.0, 25.5],
    'Rainfall (mm)': [2200, 750, 700, 1100, 725, 1000],
    'Population Density': [20700, 9500, 5700, 10600, 8700, 8100]
})
print(df)
```

```
# a) Treemap of population density
colors = ['#e63946', '#f4a261', '#2a9d8f', '#457b9d', '#8ab17d', '#e9c46a']
labels = [f"{c}\n{d}" for c, d in zip(df['City'], df['Population Density'])]
plt.figure(figsize=(9, 6))
squarify.plot(sizes=df['Population Density'], label=labels,
              color=colors, alpha=0.85, text_kwargs={'fontsize': 11})
plt.title('Treemap - Population Density of Cities (people per sq. km)')
plt.axis('off')
plt.show()

# b) 3D Scatter plot
fig = plt.figure(figsize=(9, 7))
ax = fig.add_subplot(111, projection='3d')
pts = ax.scatter(df['Area (sq. km)'], df['Average Temperature (C)'],
df['Rainfall (mm)'],
                 c=df['Population Density'], cmap='viridis', s=120)
for i in range(len(df)):
    ax.text(df['Area (sq. km)'][i], df['Average Temperature (C)'][i],
            df['Rainfall (mm)'][i], df['City'][i])
ax.set_xlabel('Area (sq. km)')
ax.set_ylabel('Avg Temperature (C)')
ax.set_zlabel('Rainfall (mm)')
ax.set_title('3D Scatter Plot - Area, Temperature, Rainfall')
fig.colorbar(pts, label='Population Density', shrink=0.6)
plt.show()
```

Q2 OR. Apriori – Groceries (support 0.3 and 0.4) [20]

Same program as **Slip 7 – Q2 OR**. Copy it as it is, including the explanation that only Cooking Oil (support 4/7 = 0.57) is frequent at 0.3 and 0.4.

Slip 15 – Viva

1. **What is data discretization?** Converting a continuous attribute (like Age) into a small number of intervals or categories.
2. **Equal width vs equal frequency binning?** Equal width: every bin has the same range, width = (max − min) / bins, via `pd.cut`. Equal frequency: every bin has the same number of records, via `pd.qcut`.
3. **When is equal frequency better?** When data is skewed; equal width would leave some bins nearly empty.

4. **What is `retbins=True`?** It also returns the bin edges so you can show where each bin starts and ends.
5. **What is a treemap?** Nested rectangles where each rectangle's area is proportional to its value. Good for part-to-whole with many categories.
6. **Which library makes a treemap?** `squarify` (with matplotlib); plotly also has `px.treemap`.
7. **Why a 3D scatter plot?** To see the relation among three numeric variables at once; colour adds a 4th variable.
8. **How do you make a 3D axis in matplotlib?** `fig.add_subplot(111, projection='3d')`.
9. **What does `cmap='viridis'` mean?** A colour map that turns numbers into a colour scale.

Slip 16 (Page 18)

Q1. Correlation Matrix + Heat Map – Student Performance [10]

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.DataFrame({
    'Study Hours': [2, 3, 4, 5, 6, 7, 8, 1],
    'Attendance': [65, 70, 78, 80, 88, 92, 95, 60],
    'Assignment Score': [55, 60, 66, 72, 80, 85, 92, 50],
    'Exam Score': [48, 58, 62, 70, 82, 86, 94, 40]
})
print(df)

corr = df.corr()            # Pearson correlation by default
print('\nCorrelation Matrix:')
print(corr.round(2))

plt.figure(figsize=(7, 5))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            vmin=-1, vmax=1, linewidths=0.5)
plt.title('Correlation Heat Map - Student Performance')
plt.show()
```

All values come out close to +1, meaning more study hours and attendance go with higher scores.

Q2. Min-Max Scaling, Standardization, Normalization – Weather Data [20]

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler, Normalizer

df = pd.DataFrame({
    'Temperature': [22, 25, 30, 35, 40, 28],
    'Humidity': [45, 50, 62, 70, 82, 55],
    'Wind Speed': [5, 8, 10, 12, 15, 7],
    'Pressure': [1002, 1005, 1010, 1013, 1020, 1008],
    'Rainfall': [10, 20, 35, 40, 55, 25]
})
print('Original Data:\n', df)

# a) Min-Max Scaling -> values between 0 and 1 (column-wise)
minmax = pd.DataFrame(MinMaxScaler().fit_transform(df), columns=df.columns)
print('\nMin-Max Scaled:\n', minmax.round(3))

# b) Standardization -> mean 0, std 1 (column-wise)
standard = pd.DataFrame(StandardScaler().fit_transform(df),
columns=df.columns)
print('\nStandardized:\n', standard.round(3))
print('Mean:', standard.mean().round(2).tolist())
print('Std :', standard.std(ddof=0).round(2).tolist())

# c) Normalization -> each ROW has length 1 (L2 norm)
normal = pd.DataFrame(Normalizer(norm='l2').fit_transform(df),
columns=df.columns)
print('\nNormalized:\n', normal.round(4))
```

Formulas to write in the journal:

```
x_{minmax} = \frac{x - x_{min}}{x_{max} - x_{min}} \qquad z = \frac{x - \mu}
{\sigma} \qquad x_{norm} = \frac{x}{\sqrt{x_1^2 + x_2^2 + \dots + x_n^2}}
```

Q2 OR. Apriori – Student Activity Dataset (support 0.2 and 0.3) [20]

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [
    ['Library', 'Seminar'],
    ['Workshop', 'Project'],
    ['Library', 'Assignment'],
    ['Seminar', 'Workshop'],
    ['Assignment', 'Seminar'],
    ['Workshop', 'Assignment'],
    ['Project', 'Library'],
    ['Library', 'Seminar', 'Project'],
    ['Library', 'Seminar', 'Workshop']
]

te = TransactionEncoder()
df = pd.DataFrame(te.fit_transform(transactions), columns=te.columns_)
print(df.astype(int))

for support in [0.2, 0.3]:
    print('\n===== Minimum Support =', support, '=====')
    freq = apriori(df, min_support=support, use_colnames=True)
    print('Frequent Itemsets:\n', freq)
    try:
        rules = association_rules(freq, num_itemsets=len(df),
                                  metric='confidence', min_threshold=0.5)
    except TypeError:        # older mlxtend versions
        rules = association_rules(freq, metric='confidence',
min_threshold=0.5)
    print('Association Rules:\n',
          rules[['antecedents', 'consequents', 'support', 'confidence',
'lift']])
```

Expected output (9 transactions):

Min support	Frequent itemsets	Rules (confidence ≥ 0.5)
0.2	All 5 single items; {Library, Seminar} 0.33; {Library, Project} 0.22; {Seminar, Workshop} 0.22	Library → Seminar 0.60; Seminar → Library 0.60; Project → Library 0.67; Workshop → Seminar 0.50
0.3	All 5 single items (Library 0.56, Seminar 0.56, Workshop 0.44, Project 0.33, Assignment 0.33); {Library, Seminar} 0.33	Library → Seminar 0.60; Seminar → Library 0.60 (lift 1.08)

Higher support removes weaker pairs, so fewer rules survive at 0.3.

Slip 16 – Viva

1. **What is correlation?** A number from −1 to +1 showing how two variables move together; +1 perfect positive, −1 perfect negative, 0 no linear relation.
2. **Which correlation does `df.corr()` use?** Pearson by default; Spearman and Kendall are also available.
3. **Why is the diagonal of the heat map 1?** Every variable is perfectly correlated with itself.
4. **Does correlation mean causation?** No. Two variables can move together because of a third factor.
5. **Min-Max vs Standardization?** Min-Max squeezes values into [0, 1] and is affected by outliers; Standardization gives mean 0, std 1 and handles outliers better.
6. **How is Normalizer different from the other two?** Normalizer works on each row (each record gets unit length); Min-Max and Standard scaling work on each column.
7. **When is scaling necessary?** For distance-based algorithms (KNN, K-Means), gradient-based models and PCA.
8. **What is a frequent itemset?** A set of items whose support is at least the minimum support.
9. **Why is {Library, Seminar} the strongest pair?** It appears in 3 of 9 transactions, the highest of any pair.
10. **What does lift 1.08 mean?** Students who visit the library are slightly more likely (8%) than average to attend seminars.

Slip 17 (Page 19)

Q1. Data Integration – student_details + academic_records [10]

```
import pandas as pd

student_details = pd.DataFrame({
    'Student_ID': [101, 102, 103, 104, 105],
    'Name': ['Aarav', 'Diya', 'Kabir', 'Meera', 'Rohan'],
    'Gender': ['M', 'F', 'M', 'F', 'M'],
    'City': ['Nashik', 'Pune', 'Mumbai', 'Nashik', 'Nagpur']
})
academic_records = pd.DataFrame({
    'Student_ID': [101, 102, 103, 104, 106],
    'Course': ['B.Sc CS', 'B.Sc CS', 'BCA', 'B.Sc CS', 'BCA'],
    'Marks': [82, 76, 91, 68, 74],
    'Attendance': [90, 85, 95, 72, 80]
})
print('student_details:\n', student_details)
print('\nacademic_records:\n', academic_records)

integrated = pd.merge(student_details, academic_records, on='Student_ID',
how='inner')
print('\nIntegrated DataFrame (inner join):\n', integrated)

outer = pd.merge(student_details, academic_records, on='Student_ID',
how='outer')
print('\nOuter join (shows unmatched IDs with NaN):\n', outer)
```

Student 105 has no academic record and 106 has no details, so the inner join keeps only 101–104.

Q2. Treemap (Population) + 3D Scatter (Population, GDP, Literacy) [20]

```
import pandas as pd
import matplotlib.pyplot as plt
import squarify

df = pd.DataFrame({
    'City': ['Mumbai', 'Delhi', 'Bengaluru', 'Pune', 'Nagpur', 'Nashik'],
    'Population': [12.4, 11.0, 8.4, 3.1, 2.4, 1.5],        # millions
    'GDP': [310, 290, 110, 69, 30, 20],                    # billion USD
```

```
    'Literacy Rate': [89.7, 86.2, 88.7, 89.6, 91.9, 88.4]  # percent
})
print(df)

# a) Treemap
labels = [f"{c}\n{p}M" for c, p in zip(df['City'], df['Population'])]
plt.figure(figsize=(9, 6))
squarify.plot(sizes=df['Population'], label=labels,
              color=plt.cm.Set2.colors[:len(df)], alpha=0.9)
plt.title('Treemap - City-wise Population Distribution')
plt.axis('off')
plt.show()

# b) 3D Scatter
fig = plt.figure(figsize=(9, 7))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(df['Population'], df['GDP'], df['Literacy Rate'],
           c=df['GDP'], cmap='plasma', s=150)
for i in range(len(df)):
    ax.text(df['Population'][i], df['GDP'][i], df['Literacy Rate'][i],
df['City'][i])
ax.set_xlabel('Population (millions)')
ax.set_ylabel('GDP (billion USD)')
ax.set_zlabel('Literacy Rate (%)')
ax.set_title('3D Scatter - Population vs GDP vs Literacy Rate')
plt.show()
```

The figures are approximate sample values; that is fine because the question says create a dataset.

Q2 OR. Random Forest Classifier – Student Grading Dataset [20]

`Students_Grading_Dataset.csv` has Student_ID, First_Name, Last_Name, Email, Gender, Age, Department, Attendance (%), Midterm_Score, Final_Score, Assignments_Avg, Quizzes_Avg, Participation_Score, Projects_Score, Total_Score, Grade (A–F), Study_Hours_per_Week, and some lifestyle columns. Target = Grade.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
f1_score,
                             confusion_matrix, ConfusionMatrixDisplay,
classification_report)

# 1. Read and inspect
df = pd.read_csv('Students_Grading_Dataset.csv')
print(df.head())
print(df.info())
print(df.isnull().sum())

# 2. Preprocess
df = df.drop(columns=['Student_ID', 'First_Name', 'Last_Name', 'Email'],
errors='ignore')
for c in df.columns:                          # fill missing values
    if df[c].dtype == 'object':
        df[c] = df[c].fillna(df[c].mode()[0])
    else:
        df[c] = df[c].fillna(df[c].median())
df = df.drop_duplicates()

y = df['Grade']
X = df.drop(columns=['Grade'])
for c in X.select_dtypes(include='object').columns:
    X[c] = LabelEncoder().fit_transform(X[c])

# 3. Split and train
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
pred = rf.predict(X_test)

# 4. Evaluate
print('Accuracy :', accuracy_score(y_test, pred))
print('Precision:', precision_score(y_test, pred, average='weighted'))
print('Recall   :', recall_score(y_test, pred, average='weighted'))
print('F1 Score :', f1_score(y_test, pred, average='weighted'))
print(classification_report(y_test, pred))
cm = confusion_matrix(y_test, pred)
print('Confusion Matrix:\n', cm)
ConfusionMatrixDisplay(cm, display_labels=rf.classes_).plot()
plt.title('Random Forest - Student Grades')
```

```
plt.show()

# 5. Feature importance
imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values()
imp.plot(kind='barh', figsize=(8, 6), title='Feature Importance')
plt.show()
```

If accuracy comes out low, explain that this dataset is synthetic and grades are only loosely linked to the scores; the feature importance chart still shows which features matter most.

Slip 17 – Viva

1. **What is data integration?** Combining data from different sources into one consistent dataset.
2. **`merge` vs `concat`?** `merge` joins on a common key column (like SQL JOIN); `concat` stacks DataFrames vertically or side by side.
3. **Problems in data integration?** Different column names for the same thing, duplicate records, mismatched units and missing keys.
4. **Why did the outer join show NaN?** IDs present in only one table have no matching values from the other.
5. **What is Random Forest?** An ensemble of many decision trees; each tree votes and the majority class wins.
6. **What are bagging and bootstrap sampling?** Each tree trains on a random sample of rows taken with replacement, and a random subset of features is tried at each split.
7. **Why is Random Forest better than one decision tree?** Averaging many trees reduces overfitting and variance.
8. **What is `n_estimators`?** The number of trees in the forest (100 here).
9. **What is feature importance?** How much each feature reduces impurity (Gini) across all trees; higher = more useful.
10. **What is Gini impurity?** $1 - \Sigma\, p_i^2$. It is 0 when a node contains only one class.

Slip 18 (Page 20)

Q1. Data Cleaning – employee_data (missing, duplicates, IQR outliers) [10]

```
import pandas as pd
import numpy as np

employee_data = pd.DataFrame({
    'Emp_ID': [1, 2, 3, 3, 4, 5, 6, 7],
    'Age': [25, 30, np.nan, np.nan, 45, 38, 29, 95],              # 95 =
outlier
    'Dept': ['HR', 'IT', 'Sales', 'Sales', None, 'IT', 'Finance', 'HR'],
    'Years_Experience': [2, 5, 8, 8, 15, 10, np.nan, 12],
    'Performance_Rating': [3, 4, 5, 5, 4, 3, 4, 5]
})
print('Original Data:\n', employee_data)
print('\nMissing values:\n', employee_data.isnull().sum())

# 1. Remove duplicate records
df = employee_data.drop_duplicates()
print('\nAfter removing duplicates:', len(df), 'rows')

# 2. Handle missing values
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Years_Experience'] =
df['Years_Experience'].fillna(df['Years_Experience'].mean())
df['Dept'] = df['Dept'].fillna(df['Dept'].mode()[0])
print('\nAfter filling missing values:\n', df)

# 3. Detect outliers using IQR
for col in ['Age', 'Years_Experience', 'Performance_Rating']:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    out = df[(df[col] < lower) | (df[col] > upper)]
    print(f'\n{col}: Q1={Q1}, Q3={Q3}, IQR={IQR}, limits=({lower}, {upper})')
    print('Outliers:\n', out if len(out) else 'None')
    df = df[(df[col] >= lower) & (df[col] <= upper)]

print('\nCleaned Data:\n', df)
```

Age 95 is detected and removed as an outlier.

Q2. World Cities – Geospatial Map + Treemap [20]

`worldcities.csv` columns: city, city_ascii, lat, lng, country, iso2, iso3, admin_name, capital, population, id.

```
import pandas as pd
import plotly.express as px
import matplotlib.pyplot as plt
import squarify

# 1. Read and clean
df = pd.read_csv('worldcities.csv')
print(df.head())
print(df.isnull().sum())
df = df.dropna(subset=['lat', 'lng', 'population'])
top = df.nlargest(100, 'population')            # 100 biggest cities

# a) Geospatial visualization - marker size = population
fig = px.scatter_geo(top, lat='lat', lon='lng', size='population',
                     color='country', hover_name='city',
                     projection='natural earth',
                     title='Top 100 World Cities by Population')
fig.show()

# b) Treemap of the 20 largest cities
top20 = df.nlargest(20, 'population')
labels = [f"{c}\n{p/1e6:.1f}M" for c, p in zip(top20['city'],
top20['population'])]
plt.figure(figsize=(12, 7))
squarify.plot(sizes=top20['population'], label=labels,
              color=plt.cm.tab20.colors, alpha=0.85, text_kwargs={'fontsize':
9})
plt.title('Treemap - Population Distribution of Top 20 Cities')
plt.axis('off')
plt.show()
```

If plotly does not open in the lab, add `fig.write_html('map.html')` and open the file in a browser.

Q2 OR. Simple Linear Regression – Crop Yield from Rainfall [20]

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

# 1. Read and inspect
df = pd.read_csv('crop_yield_data.csv')
print(df.head())
print(df.isnull().sum())

rain_col = [c for c in df.columns if 'rain' in c.lower()][0]
yield_col = [c for c in df.columns if 'yield' in c.lower()][0]
print('X =', rain_col, '| y =', yield_col)

# 2. Handle missing values (fill with mean)
df[rain_col] = df[rain_col].fillna(df[rain_col].mean())
df[yield_col] = df[yield_col].fillna(df[yield_col].mean())

X = df[[rain_col]]
y = df[yield_col]

# 3. Split and train
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
print('Slope:', model.coef_[0], '| Intercept:', model.intercept_)

# 4. Evaluate
pred = model.predict(X_test)
print('MAE     :', mean_absolute_error(y_test, pred))
print('R2 Score:', r2_score(y_test, pred))

# 5. Plot
plt.scatter(X_test, y_test, color='green', label='Actual')
plt.plot(X_test, pred, color='red', linewidth=2, label='Regression line')
plt.xlabel('Rainfall')
```

```
plt.ylabel('Crop Yield')
plt.title('Rainfall vs Crop Yield')
plt.legend()
plt.show()
```

Slip 18 – Viva

1. **What is data cleaning?** Fixing or removing wrong, missing, duplicate or extreme data so analysis is reliable.
2. **Ways to handle missing values?** Drop rows/columns, or fill with mean, median, mode, a constant, or forward/backward fill.
3. **Why median for Age, not mean?** Median is not pulled by outliers like 95.
4. **What is IQR?** Q3 − Q1, the spread of the middle 50% of data.
5. **IQR outlier rule?** A value is an outlier if it is below Q1 − 1.5×IQR or above Q3 + 1.5×IQR.
6. **Other outlier methods?** Z-score (|z| > 3) and box plots.
7. **What is geospatial visualization?** Plotting data on a map using latitude and longitude.
8. **Why use plotly for maps?** `scatter_geo` draws an interactive world map with zoom and hover info in one line.
9. **Why only the top 100 cities?** The file has tens of thousands of cities; plotting all is slow and unreadable.
10. **MAE meaning?** Mean Absolute Error: average of |actual − predicted|, in the same unit as yield.
11. **What does a negative R^2 mean?** The model predicts worse than simply using the average value.

Slip 19 (Page 21)

Q1. Box Plots – Vehicle Data with Outlier [10]

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    'Fuel_Efficiency': [18, 20, 22, 21, 55],        # 55 km/L = outlier
    'Engine_Power': [90, 110, 130, 120, 400],       # 400 hp  = outlier
    'Vehicle_Weight': [1000, 1200, 1300, 1250, 1400],
    'Engine_Size': [1.2, 1.5, 1.8, 1.6, 2.0],
```

```
    'Acceleration': [12, 11, 10, 10.5, 9]
})
print(df)

fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].boxplot(df['Fuel_Efficiency'], patch_artist=True,
              boxprops=dict(facecolor='lightblue'))
ax[0].set_title('Fuel Efficiency')
ax[0].set_ylabel('km/L')
ax[1].boxplot(df['Engine_Power'], patch_artist=True,
              boxprops=dict(facecolor='lightgreen'))
ax[1].set_title('Engine Power')
ax[1].set_ylabel('hp')
plt.suptitle('Box Plots for Outlier Detection')
plt.show()

# Identify outliers with IQR
for col in ['Fuel_Efficiency', 'Engine_Power']:
    Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
    IQR = Q3 - Q1
    out = df[(df[col] < Q1 - 1.5 * IQR) | (df[col] > Q3 + 1.5 * IQR)]
    print(f'\nOutliers in {col}:')
    print(out[[col]])
```

The points shown as circles above the whiskers (55 and 400) are the outliers.

Q2. Min-Max Scaling, Standardization, Normalization [20]

Same program as **Slip 16 – Q2**. Copy it as it is (Temperature, Humidity, Wind Speed, Pressure, Rainfall with all three transformations and the formulas).

Q2 OR. Random Forest Classifier – Heart Disease [20]

`heart.csv` has 13 features (age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal) and `target`: 1 = disease present, 0 = absent.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
f1_score,
                             confusion_matrix, ConfusionMatrixDisplay,
```

```
classification_report)

# 1. Read and inspect
df = pd.read_csv('heart.csv')
print(df.head())
print(df.info())
print(df.isnull().sum())
print(df['target'].value_counts())

# 2. Preprocess
df = df.dropna().drop_duplicates()
X = df.drop(columns=['target'])
y = df['target']

# 3. Split and train
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
pred = rf.predict(X_test)

# 4. Evaluate
print('Accuracy :', accuracy_score(y_test, pred))
print('Precision:', precision_score(y_test, pred))
print('Recall   :', recall_score(y_test, pred))
print('F1 Score :', f1_score(y_test, pred))
print(classification_report(y_test, pred, target_names=['No Disease',
'Disease']))
cm = confusion_matrix(y_test, pred)
print('Confusion Matrix:\n', cm)
ConfusionMatrixDisplay(cm, display_labels=['No Disease',
'Disease']).plot(cmap='Reds')
plt.title('Random Forest - Heart Disease')
plt.show()

# 5. Most important features
print(pd.Series(rf.feature_importances_,
index=X.columns).sort_values(ascending=False))
```

Note: the popular 1025-row heart.csv has many duplicate rows. Without `drop_duplicates()` accuracy looks close to 100% because the same patients appear in both train and test. With duplicates removed (about 302 rows remain), expect roughly

0.80–0.88.

Slip 19 – Viva

1. **What does a box plot show?** The five-number summary: minimum, Q1, median, Q3, maximum, with outliers as separate dots.
2. **What are the whiskers?** Lines extending to the furthest points within 1.5×IQR of the box.
3. **Why remove or treat outliers?** They distort mean, standard deviation and regression lines.
4. **What is `patch_artist=True`?** Lets you fill the box with colour.
5. **Why is Recall important in heart disease prediction?** Missing a sick patient (false negative) is dangerous, so we want high recall.
6. **What is a false positive here?** A healthy person predicted to have heart disease.
7. **Does Random Forest need feature scaling?** No. Trees split on thresholds, so scale does not matter.
8. **What is overfitting?** The model learns training data too well, including noise, and performs poorly on new data.
9. **What does `classification_report` show?** Precision, recall, F1 and support for each class plus averages.
10. **What are the key features for heart disease here?** Usually cp (chest pain type), thalach (max heart rate), ca, oldpeak and thal.

Slip 20 (Page 22)

Q1. Standardization – Employee Attributes [10]

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

df = pd.DataFrame({
    'Age': [22, 25, 30, 35, 40, 45],
    'Salary': [20000, 28000, 40000, 52000, 61000, 75000],
    'Experience': [1, 3, 5, 8, 12, 18],
    'Working_Hours': [7, 8, 8, 9, 9, 10],
    'Performance_Score': [60, 65, 72, 80, 85, 92]
})
```

```
print('Original Data:\n', df)

scaler = StandardScaler()
std_df = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)
print('\nStandardized Data:\n', std_df.round(3))

print('\nMean of each column used:', scaler.mean_.round(2))
print('Std of each column used :', scaler.scale_.round(2))
print('Mean after :', std_df.mean().round(2).tolist())          # all 0
print('Std after  :', std_df.std(ddof=0).round(2).tolist())     # all 1
```

Formula: z = (x − mean) / standard deviation.

Q2. Books Dataset – Venn Diagram of Authors + Word Cloud of Titles [20]

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib_venn import venn2
from wordcloud import WordCloud

books = pd.DataFrame({
    'Book Title': ['Python Programming Basics', 'Data Science Handbook',
'Learning Python',
                   'Machine Learning with Python', 'Deep Learning
Fundamentals',
                   'Data Analytics Made Easy', 'Artificial Intelligence
Guide',
                   'Python for Data Analysis', 'Statistics for Data Science',
                   'Machine Learning Projects'],
    'Author': ['Mark Lutz', 'Jake VanderPlas', 'Mark Lutz', 'Sebastian
Raschka',
               'Ian Goodfellow', 'Jake VanderPlas', 'Stuart Russell',
               'Wes McKinney', 'Peter Bruce', 'Sebastian Raschka'],
    'Publication Year': [2008, 2012, 2013, 2015, 2016, 2018, 2019, 2020,
2021, 2023],
    'Average Rating': [4.1, 4.5, 4.2, 4.6, 4.4, 4.0, 4.3, 4.7, 4.2, 4.5],
    'Number of Ratings': [1200, 3400, 2100, 2900, 3900, 800, 1500, 4200, 950,
1100]
})
print(books)

# a) Venn diagram: authors before 2016 vs 2016 and after
period1 = set(books[books['Publication Year'] < 2016]['Author'])
```

```
period2 = set(books[books['Publication Year'] >= 2016]['Author'])
print('2008-2015 authors:', period1)
print('2016-2023 authors:', period2)
print('Authors in both periods:', period1 & period2)

plt.figure(figsize=(7, 6))
venn2([period1, period2], set_labels=('Published 2008-2015', 'Published 2016-
2023'),
      set_colors=('orange', 'teal'), alpha=0.7)
plt.title('Authors in Two Time Periods')
plt.show()

# b) Word cloud from book titles
text = ' '.join(books['Book Title'])
wc = WordCloud(width=800, height=400, background_color='white',
               colormap='viridis').generate(text)
plt.figure(figsize=(10, 5))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud of Book Titles')
plt.show()
```

In the word cloud, Python, Data and Learning appear biggest because they occur most often in titles. Jake VanderPlas and Sebastian Raschka sit in the overlap of the Venn diagram.

Q2 OR. Apriori – Student Activity Dataset (support 0.2 and 0.3) [20]

Same program as **Slip 16 – Q2 OR**. Copy it as it is, along with its expected-output table.

Slip 20 – Viva

1. **What is standardization?** Rescaling each column to mean 0 and standard deviation 1 (z-score).
2. **Can standardized values be negative?** Yes. Values below the mean become negative.
3. **Does standardization change the shape of the distribution?** No, it only shifts and rescales it.
4. **Which algorithms need standardization?** KNN, K-Means, SVM, PCA, Logistic Regression with regularization.
5. **What is a word cloud?** A picture of words where bigger words appear more often in the text.

6. **What are stopwords?** Common words like 'the', 'for', 'with' that WordCloud removes by default.
7. **What does `interpolation='bilinear'` do?** Smooths the image so the words look sharp.
8. **What does the overlap in the Venn diagram mean?** Authors who published books in both time periods.
9. **Why use sets for Venn diagrams?** Sets remove duplicate names and support union, intersection and difference.
10. **venn2 vs venn3?** venn2 compares two sets, venn3 compares three.

Slip 22 (Page 24)

Q1. Normalization – House Rent Data [10]

```
import pandas as pd
from sklearn.preprocessing import Normalizer

df = pd.DataFrame({
    'City': ['Nashik', 'Pune', 'Mumbai', 'Nagpur', 'Aurangabad', 'Pune'],
    'House_Type': ['1BHK', '3BHK', '2BHK', '2BHK', '3BHK', '1BHK'],
    'House_Area': [550, 1450, 900, 950, 1300, 600],          # sq. ft
    'Number_of_Rooms': [1, 3, 2, 2, 3, 1],
    'Monthly_Rent': [8000, 32000, 45000, 14000, 16000, 15000]
})
print('Original Data:\n', df)

num_cols = ['House_Area', 'Number_of_Rooms', 'Monthly_Rent']   # only numeric
columns
norm_df = df.copy()
norm_df[num_cols] = Normalizer(norm='l2').fit_transform(df[num_cols])
print('\nNormalized Data:\n', norm_df.round(4))

# Check: every row now has length 1
print('\nRow lengths:', (norm_df[num_cols] **
2).sum(axis=1).round(2).tolist())
```

City and House_Type are text, so they are left unchanged. Note: if the external expects Min-Max as "normalization", say both terms are used, and show `MinMaxScaler` too (code in Slip 16 – Q2).

Q2. Netflix & Amazon Prime – Venn Diagram + Pie Chart [20]

Same program as **Slip 5 – Q2**. Copy it as it is.

Q2 OR. K-Means (K = 3) – Student Attendance and Marks [20]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# 1. Dataset (with one missing value to show preprocessing)
df = pd.DataFrame({
    'Student': [f'S{i}' for i in range(1, 16)],
    'Attendance': [55, 60, 62, 68, 70, 72, 75, 78, 82, 85, 88, 90, 93, 95,
np.nan],
    'Marks':      [35, 40, 45, 50, 52, 58, 60, 62, 70, 75, 80, 84, 88, 92,
66]
})
print(df)
print(df.isnull().sum())

# 2. Preprocess: fill missing, scale
df['Attendance'] = df['Attendance'].fillna(df['Attendance'].mean())
scaler = StandardScaler()
X = scaler.fit_transform(df[['Attendance', 'Marks']])

# 3. K-Means with K = 3
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X)

# name clusters by average marks
order = df.groupby('Cluster')['Marks'].mean().sort_values().index
df['Group'] = df['Cluster'].map(dict(zip(order, ['Weak', 'Average',
'Good'])))
print(df)

# 4. Visualize clusters and centroids
centers = scaler.inverse_transform(kmeans.cluster_centers_)
print('Centroids (Attendance, Marks):\n', centers.round(2))

plt.figure(figsize=(8, 6))
```

```
for g, col in zip(['Weak', 'Average', 'Good'], ['red', 'orange', 'green']):
    part = df[df['Group'] == g]
    plt.scatter(part['Attendance'], part['Marks'], c=col, s=80, label=g)
plt.scatter(centers[:, 0], centers[:, 1], c='black', marker='X', s=250,
label='Centroids')
plt.xlabel('Attendance (%)')
plt.ylabel('Marks')
plt.title('K-Means Clustering of Students (K = 3)')
plt.legend()
plt.show()
```

Slip 22 – Viva

1. **What is normalization (Normalizer)?** Scaling each row so its vector length (L2 norm) is 1.
2. **L1 vs L2 norm?** L1 = sum of absolute values; L2 = square root of sum of squares.
3. **Why not normalize City and House_Type?** They are text (categorical); scaling only works on numbers. Encode them first if needed.
4. **Where is row normalization useful?** Text data (TF-IDF vectors) and cosine-similarity based methods.
5. **Why scale Attendance and Marks before K-Means?** So both features contribute equally to the distance.
6. **Why use `inverse_transform` for centroids?** To plot centroids back in the original units (% and marks).
7. **What is `n_init=10`?** K-Means runs 10 times with different starting centroids and keeps the best result.
8. **Why `random_state=42`?** So the same clusters appear every time the code runs.
9. **How would you label the clusters?** By their averages: low attendance + low marks = Weak, middle = Average, high = Good.
10. **K-Means vs KNN?** K-Means is unsupervised clustering (K = number of clusters); KNN is supervised classification (K = number of neighbours).

Slip 23 (Page 25)

Q1. Dendrogram with Product Names [10]

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from sklearn.preprocessing import StandardScaler

df = pd.DataFrame({
    'Product_name': ['Mixer', 'Laptop', 'Earbuds', 'Smart TV', 'Kettle',
                     'Phone', 'Charger', 'Refrigerator'],
    'Price': [3500, 55000, 1500, 42000, 1200, 18000, 600, 38000],
    'Rating': [4.1, 4.5, 4.0, 4.4, 3.9, 4.3, 3.8, 4.2],
    'Number_of_Sales': [800, 250, 2200, 300, 1500, 1100, 3000, 200]
})
print(df)

X = StandardScaler().fit_transform(df[['Price', 'Rating',
'Number_of_Sales']])
Z = linkage(X, method='ward')

plt.figure(figsize=(10, 6))
dendrogram(Z, labels=df['Product_name'].tolist(), leaf_rotation=45)
plt.axhline(y=3, color='red', linestyle='--', label='Cut line')
plt.title('Dendrogram - Groups of Similar Products')
plt.xlabel('Product Name')
plt.ylabel('Distance (Ward)')
plt.legend()
plt.show()

# Groups after cutting the tree into 3 clusters
df['Group'] = fcluster(Z, t=3, criterion='maxclust')
print(df[['Product_name', 'Group']].sort_values('Group'))
```

Expected grouping: expensive, low-sales items (Laptop, Smart TV, Refrigerator) together, cheap high-sales items (Earbuds, Charger, Kettle) together, and mid-range items in between.

Q2. Min-Max Scaling, Standardization, Normalization [20]

Same program as **Slip 16 – Q2**. Copy it as it is.

Q2 OR. KNN Classifier – Student Grading Dataset [20]

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
f1_score,
                             confusion_matrix, ConfusionMatrixDisplay)

# 1. Read and inspect
df = pd.read_csv('Students_Grading_Dataset.csv')
print(df.head())
print(df.isnull().sum())

# 2. Preprocess
df = df.drop(columns=['Student_ID', 'First_Name', 'Last_Name', 'Email'],
errors='ignore')
for c in df.columns:
    if df[c].dtype == 'object':
        df[c] = df[c].fillna(df[c].mode()[0])
    else:
        df[c] = df[c].fillna(df[c].median())
df = df.drop_duplicates()

y = df['Grade']
X = df.drop(columns=['Grade'])
for c in X.select_dtypes(include='object').columns:
    X[c] = LabelEncoder().fit_transform(X[c])

# 3. Split and scale (KNN needs scaling)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Find a good K
for k in [3, 5, 7, 9, 11]:
    acc = KNeighborsClassifier(n_neighbors=k).fit(X_train,
y_train).score(X_test, y_test)
    print('K =', k, 'accuracy =', round(acc, 3))
```

```
# 5. Final model with K = 5
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
pred = knn.predict(X_test)

print('Accuracy :', accuracy_score(y_test, pred))
print('Precision:', precision_score(y_test, pred, average='weighted',
zero_division=0))
print('Recall   :', recall_score(y_test, pred, average='weighted'))
print('F1 Score :', f1_score(y_test, pred, average='weighted'))
cm = confusion_matrix(y_test, pred)
print('Confusion Matrix:\n', cm)
ConfusionMatrixDisplay(cm, display_labels=knn.classes_).plot()
plt.title('KNN - Student Grades')
plt.show()
```

Slip 23 – Viva

1. **Why scale before making a dendrogram?** Price is in thousands and Rating is 1–5; without scaling the distances depend only on Price.
2. **What does `fcluster` do?** Cuts the dendrogram to give each item a cluster number.
3. **Hierarchical clustering vs K-Means?** Hierarchical needs no K in advance and gives a tree, but is slow for big data; K-Means is fast but needs K.
4. **Single vs complete linkage?** Single uses the closest pair of points between clusters; complete uses the farthest pair.
5. **Why try several K values in KNN?** Accuracy changes with K; we pick the K with the best test accuracy.
6. **What is `zero_division=0`?** Avoids a warning when a class gets no predictions, by treating its precision as 0.
7. **Why drop names and Email?** They are unique identifiers with no predictive value and would confuse the model.
8. **Is KNN good for large datasets?** Not really; prediction is slow because it compares with every training point.
9. **What is macro vs weighted average?** Macro averages all classes equally; weighted gives larger classes more weight.
10. **Can KNN handle text columns directly?** No. Text must be encoded into numbers first.

Slip 25 (Page 27)

Q1. Bar Chart – Students Enrolled in Courses [10]

Same program as **Slip 7 – Q1**. Copy it as it is (different bar colours, title, both axes labelled, values on top of bars).

Q2. Data Integration + Label Encoding + One-Hot Encoding (Mobiles) [20]

Same program as **Slip 14 – Q2 OR**. Copy it as it is; it displays both original DataFrames, the merged DataFrame and the transformed DataFrame.

Q2 OR. Apriori – Groceries (support 0.3 and 0.4) [20]

Same program as **Slip 7 – Q2 OR**. Copy it as it is, including the explanation of the result.

Slip 25 – Viva

1. **What is a label in a bar chart?** Text on the axes (`xlabel` , `ylabel`) telling what each axis measures.
2. **Vertical vs horizontal bar chart?** `plt.bar` draws vertical bars; `plt.barh` draws horizontal bars, better for long category names.
3. **What is One-Hot Encoding's drawback?** Many categories create many columns (curse of dimensionality).
4. **What is the dummy variable trap?** One-Hot columns are perfectly correlated; drop one with `drop_first=True` for linear models.
5. **get_dummies vs OneHotEncoder?** `pd.get_dummies` is quick for a DataFrame; sklearn's `OneHotEncoder` remembers categories, so it can be reused on test data.
6. **Which key did you merge on and why?** Model, because it is the common column present in both DataFrames.
7. **Why does Apriori become slow on large data?** It scans the database many times and the number of candidate itemsets grows quickly.
8. **Alternative to Apriori?** FP-Growth, which builds a tree and avoids generating candidates (`mlxtend.frequent_patterns.fpgrowth`).
9. **Why is Cooking Oil the only frequent item?** It appears in 4 of 7 baskets (0.57); everything else appears at most twice (0.29).
10. **What business decision can come from a rule like Onions → Cooking Oil?** Keep them near each other or offer a combo discount.

General Viva (asked in any slip)

1. **What is Data Science?** Extracting useful knowledge from data using statistics, programming and domain knowledge.
2. **Steps of a data science project?** Collect data → clean/preprocess → explore and visualize → build model → evaluate → deploy and communicate results.
3. **Supervised vs unsupervised learning?** Supervised has a target label (regression, classification: Linear/Logistic Regression, KNN, Random Forest). Unsupervised has no label (K-Means, hierarchical clustering, Apriori).
4. **Regression vs classification?** Regression predicts a number (salary, price); classification predicts a category (spam/not spam).
5. **What is data preprocessing?** Cleaning (missing values, duplicates, outliers), transformation (scaling, encoding), integration, reduction and discretization.
6. **What is data reduction?** Reducing data size while keeping information: feature selection, PCA (feature extraction), sampling.
7. **What is PCA?** Principal Component Analysis turns many correlated features into a few uncorrelated components that keep most of the variance.
8. **Why split into train and test?** To check performance on unseen data and detect overfitting.
9. **What is data leakage?** When test information sneaks into training, for example fitting the scaler on all data. Always fit on training data only.
10. **Underfitting vs overfitting?** Underfitting: model too simple, poor on both train and test. Overfitting: great on train, poor on test.
11. **Common regression metrics?** MAE (average absolute error), MSE (average squared error), RMSE (√MSE, same unit as target), R^2 (variance explained).
12. **Common classification metrics?** Accuracy, Precision, Recall, F1-score, Confusion Matrix.
13. **TP, TN, FP, FN?** TP: predicted yes, actually yes. TN: predicted no, actually no. FP: predicted yes, actually no (Type I error). FN: predicted no, actually yes (Type II error).
14. **Why is accuracy misleading on imbalanced data?** If 95% emails are not spam, predicting 'not spam' always gives 95% accuracy but catches no spam.
15. **pandas vs numpy?** numpy works with fast numeric arrays; pandas adds labelled tables (DataFrame) for real-world data.
16. **Series vs DataFrame?** Series is one column (1-D); DataFrame is a table of many columns (2-D).

17. **Useful pandas functions?** `head()`, `info()`, `describe()`, `isnull().sum()`, `dropna()`, `fillna()`, `drop_duplicates()`, `groupby()`, `merge()`, `value_counts()`.
18. **matplotlib vs seaborn vs plotly?** matplotlib: basic static plots. seaborn: built on matplotlib, nicer statistical plots like heatmaps. plotly: interactive charts and maps.
19. **Which chart for what?** Bar: compare categories. Pie: parts of a whole. Histogram: distribution. Box plot: spread and outliers. Scatter: relation between two variables. Heat map: correlation matrix. Treemap: hierarchy or part-to-whole. Venn: set overlap. Word cloud: word frequency.
20. **What is the role of `random_state`?** It fixes the randomness so results are reproducible.
21. **What is `train_test_split(test_size=0.2)`?** 80% training, 20% testing.
22. **What is cross-validation?** Splitting data into K folds and training K times, each time testing on a different fold, for a more reliable score.
23. **What is the curse of dimensionality?** With too many features, data becomes sparse and distance-based methods like KNN perform poorly.
24. **Which Python libraries did you use?** pandas, numpy, matplotlib, seaborn, scikit-learn, scipy, mlxtend, matplotlib-venn, wordcloud, squarify, plotly.